



CO

CONTROLE OBRAS

Arquitetura de Base de Dados

Documento tecnico para equipa de backend

Agosto 2026

Aplicacao	Controle Obras (Android)
Package	pt.controleobras.app
BD Local	Room (SQLite) - controle_obras.db
BD Remota	MariaDB (PHPRetailConcept)
Ligacao	JDBC direto (sem API REST)
Encriptacao config	AES-256-CBC (chave XOR-ofuscada)
Versao BD local	5 (4 migracoes)

Indice

1. Visao geral da arquitetura
2. Ficheiro de configuracao XML (CONTROLE_OBRAS_CONFIG.xml)
3. Ligacao JDBC ao MariaDB
4. Base de dados remota - tabela FREF (Centros de Custo)
5. Base de dados local - Room (SQLite)
6. Tabela 'talao' - esquema completo
7. Tabela 'centro_custo' - cache local
8. TypeConverters e serializacao
9. Historico de migracoes (v1 a v5)
10. Fluxo de sincronizacao
11. Exportacao de dados (JSON, XML, CSV)
12. Informacoes para a equipa de backend

1. Visao geral da arquitetura

A aplicacao Controle Obras utiliza **duas bases de dados**: uma local no dispositivo (Room/SQLite) e uma remota no servidor da empresa (MariaDB). A comunicacao com o servidor e feita via **JDBC direto** (sem camada API REST intermédia).

TABLET ANDROID		SERVIDOR OBRA
Room (SQLite) controle_obras.db - talao - centro_custo ConfigManager - Le config XML - Descripta AES FrefRepository - Sync centros custo - Cache local Room	JDBC direto (MariaDB Client 1.8.0) Porta: 3306 Sem SSL Cleartext ----->	MariaDB PHPRetailConcept - FREF (centros custo) . FREF (PK) . NMFREF . AGNOME . ENCSERIE . SINCTAB User: (encriptado) Pass: (encriptado)

IMPORTANTE para o backend: Atualmente a app liga-se diretamente ao MariaDB via JDBC. Nao existe camada API REST. O utilizador da BD e a password estao encriptados no ficheiro XML embutido na app. Futuramente esta prevista a criacao de uma API REST para substituir o JDBC direto.

2. Ficheiro de configuracao XML

O ficheiro `CONTROLE_OBRAS_CONFIG.xml` esta embutido nos assets da APK. Contem as credenciais de acesso ao MariaDB, encriptadas com AES-256-CBC. E o mesmo formato e a mesma chave utilizada pela aplicacao Zebra anterior (sistema legado C#).

Estrutura do XML

```
<?xml version="1.0" encoding="UTF-8"?> <ZEBRA> <ZEID>C0001</ZEID> <VERSAO>01</VERSAO> <EMPRESA>  
<Id>RETAIL CONCEPT SA</Id> <NIF>508067057</NIF> <BD> <SERVIDOR>(encriptado Base64)</SERVIDOR>  
<LOGIN>(encriptado Base64)</LOGIN> <PASSWORD>(encriptado Base64)</PASSWORD> <PORTA>(encriptado  
Base64)</PORTA> <BASEDADOS>(encriptado Base64)</BASEDADOS> </BD> <TIPOLIGACAO> <ONLINE>1</ONLINE>  
<TXT>0</TXT> </TIPOLIGACAO> </EMPRESA> </ZEBRA>
```

Campos do XML

Campo	Descricao	Valor/Formato
ZEID	Identificador do dispositivo/config	C0001
VERSAO	Versao do ficheiro de config	01
Id	Nome da empresa	RETAIL CONCEPT SA
NIF	NIF da empresa (texto limpo)	508067057
SERVIDOR	IP/hostname do MariaDB (encriptado)	AES-256-CBC + Base64
LOGIN	Username da BD (encriptado)	AES-256-CBC + Base64
PASSWORD	Password da BD (encriptado)	AES-256-CBC + Base64
PORTA	Porta do MariaDB (encriptado)	AES-256-CBC + Base64
BASEDADOS	Nome da BD (encriptado)	AES-256-CBC + Base64

ONLINE	Modo de ligacao (1=online, 0=offline)	1
--------	---------------------------------------	---

Encriptacao AES-256-CBC

Os campos da seccao <BD> estao encriptados com AES-256-CBC. A app descripta em runtime.

Parametro	Valor
Algoritmo	AES-256-CBC (PKCS5Padding)
Tamanho chave	32 bytes (256 bits)
IV (Initialization Vector)	16 bytes a zero (fixo, por compatibilidade C#)
Encoding saida	Base64
Ofuscacao chave no APK	XOR com mascara 0x5A (cada byte)
Origem	Mesmo sistema da app Zebra anterior (ConfigManager.cs)

Para o backend: Se precisarem de gerar um novo ficheiro de config, os campos SERVIDOR, LOGIN, PASSWORD, PORTA e BASEDADOS devem ser encriptados com a mesma chave AES-256-CBC e depois codificados em Base64. O IV e sempre 16 bytes a zero. A chave e a mesma que era usada no sistema Zebra/C#.

3. Ligacao JDBC ao MariaDB

A app liga-se diretamente ao MariaDB usando o driver JDBC `org.mariadb.jdbc.Driver` versao 1.8.0. Nao existe pool de ligacoes — cada operacao cria uma ligacao nova.

Detalhes da ligacao

Parametro	Valor
Driver	org.mariadb.jdbc.Driver (MariaDB Java Client)
Versao driver	1.8.0
Formato URL	jdbc:mariadb://{servidor}:{porta}/{baseDados}
SSL	Desativado (servidor nao suporta)
Cleartext	android:usesCleartextTraffic=true
Pool de ligacoes	Nao (uma ligacao por operacao)
Thread	Sempre em Dispatchers.IO (coroutine)
Registo driver	Class.forName('org.mariadb.jdbc.Driver') - explicito

Codigo de ligacao (RemoteDatabaseManager.kt)

```
Class.forName("org.mariadb.jdbc.Driver") val config = configManager.obterConfig() val url =  
"jdbc:mariadb://{config.servidor}: {config.porta}/{config.baseDados}"  
DriverManager.getConnection(url, config.login, config.password)
```

NOTA: A versao 2.7.14 do MariaDB Java Client foi testada mas causou incompatibilidades com o runtime Android. Por isso a app usa a versao 1.8.0. Se o backend ativar SSL no servidor MariaDB, podemos reativar os parametros `useSSL=true&trustServerCertificate=true` no URL JDBC.

4. Base de dados remota - tabela FREF

A unica tabela que a app le do MariaDB e a **FREF** (centros de custo / obras) na base de dados **PHPRetailConcept**. O utilizador JDBC tem acesso cross-database.

Query SQL executada pela app

```
SELECT FREF, NMFREF, AGNOME FROM PHPRetailConcept.FREF WHERE ENCSERIE = ? OR SINCTAB = 1 ORDER BY  
FREF ASC
```

Colunas lidas pela app

Coluna	Tipo	Papel	Descricao
FREF	VARCHAR / TEXT	PK	Codigo do centro de custo / obra (ex: 'OBR001')
NMFREF	VARCHAR / TEXT	-	Nome completo da obra (ex: 'Obra Rua Augusta')
AGNOME	VARCHAR / TEXT	-	Abreviatura / agnome do centro de custo
ENCSERIE	VARCHAR / TEXT	Filtro	Serie do dispositivo (Android ID formatado 'AID_XXXX')
SINCTAB	INT / TINYINT	Filtro	Flag de sincronizacao (1 = sincronizar para todos)

Parametro de filtragem

O parametro ? na query e preenchido com o **Android ID** do dispositivo, formatado como AID_XXXX. Isto permite que cada tablet so veja as obras que lhe foram atribuidas (via coluna ENCserie). Registos com SINCTAB = 1 sao visiveis por todos os dispositivos.

Para o backend: Se quiserem atribuir obras a um tablet especifico, basta inserir o Android ID do tablet na coluna ENCserie dos registos FREF correspondentes. O formato e 'AID_' seguido do Settings.Secure.ANDROID_ID do dispositivo. Registos com SINCTAB=1 aparecem em todos os tablets.

5. Base de dados local - Room (SQLite)

A base de dados local é gerida pelo **Room** (ORM oficial do Android sobre SQLite). O ficheiro é `controle_obras.db`, versão atual **5** (4 migrações desde v1).

Parametro	Valor
Ficheiro	controle_obras.db
Versão atual	5
ORM	Room 2.8.4
Tabelas	talao, centro_custo
TypeConverters	LocalDate, LocalTime, Instant, BigDecimal, List<ItemTalaoDto> (JSON)
Migrações	MIGRATION_1_2, MIGRATION_2_3, MIGRATION_3_4, MIGRATION_4_5
Injeção	Hilt (DatabaseModule - @Singleton)
Exportar schema	Sim (exportSchema = true)

6. Tabela 'talao' - esquema completo

Tabela principal da app. Cada linha é uma fatura/talão digitalizado. Os dados são preenchidos pelo OCR e validados pelo utilizador.

Coluna	Tipo SQL	Restrição	Desde	Descrição
id	INTEGER	PK auto	v1	ID único (auto-increment)
empresa	TEXT	NOT NULL	v1	Nome do fornecedor/emittente
nif	TEXT	NULL	v1	NIF do fornecedor (9 dígitos)
nifCliente	TEXT	NULL	v3	NIF do cliente (empresa de construção)
morada	TEXT	NULL	v1	Morada do emittente
data	TEXT	NULL	v1	Data emissão (ISO: yyyy-MM-dd)
hora	TEXT	NULL	v1	Hora emissão (ISO: HH:mm:ss)
serie	TEXT	NULL	v3	Série do doc AT (ex: 'A' de 'FT A/1234')
numeroFatura	TEXT	NULL	v1	Número da fatura (ex: 'FT 123/2024')
dataVencimento	TEXT	NULL	v3	Data de vencimento (ISO)
metodoPagamento	TEXT	NULL	v3	Método pagamento (MB, Numerário...)
itens	TEXT	NOT NULL	v1	JSON array de ItemTalaoDto (ver abaixo)
iva	TEXT	NULL	v1	Total IVA (BigDecimal como texto)
total	TEXT	NULL	v1	Total com IVA (BigDecimal como texto)
observacoes	TEXT	NULL	v1	Notas livres do utilizador
imagemPath	TEXT	NOT NULL	v1	Caminho absoluto da imagem no disco
criadoEm	TEXT	NOT NULL	v1	Timestamp UTC (ISO Instant)
textoOcr	TEXT	NULL	v2	Texto completo do OCR (debug/pesquisa)
funcn	TEXT	NOT NULL "	v5	Número do funcionário
fref	TEXT	NOT NULL "	v5	Código do centro de custo
nmfref	TEXT	NOT NULL "	v5	Nome do centro de custo
agnome	TEXT	NOT NULL "	v5	Abreviatura do centro de custo

Estrutura JSON da coluna 'itens'

A coluna itens contem um array JSON serializado com os produtos da fatura. Cada item tem a seguinte estrutura:

Campo JSON	Tipo	Descrição
descricao	String	Nome do produto/servico
quantidade	String	Quantidade (ex: '2', '1.500')
precoUnitario	String	Preco unitario (ex: '12.50')
desconto	String	Desconto na linha (vazio se nao houver)
taxalva	String	Taxa IVA em % (ex: '23', '6')
total	String	Total da linha (qtd x preco - desconto)

Exemplo: [{"descricao":"Cimento Portland","quantidade":"5",
"precoUnitario":"8.50","desconto":"","taxaIva":"23", "total":"42.50"}, {"descricao":"Areia fina
25kg","quantidade":"10", "precoUnitario":"3.20","desconto":"","taxaIva":"23", "total":"32.00"}]

7. Tabela 'centro_custo' - cache local

Cache local da tabela FREF do MariaDB. Populada a cada sincronizacao. Permite que a app funcione offline apos a primeira sincronizacao.

Coluna	Tipo SQL	Restricao	Descricao
fref	TEXT	PRIMARY KEY	Codigo do centro de custo (igual ao FREF remoto)
nmfref	TEXT	NOT NULL	Nome completo da obra
agnome	TEXT	NOT NULL	Abreviatura

A sincronizacao e um **replace completo**: apaga todos os registos locais e insere os novos (operacao atomica via substituirTodos()).

8. TypeConverters e serializacao

O Room nao suporta nativamente todos os tipos Kotlin usados. Os seguintes conversores estao registados:

Tipo Kotlin	Tipo SQLite	Conversao	Exemplo
LocalDate	TEXT (ISO)	toString() / parse()	'2026-07-23'
LocalTime	TEXT (ISO)	toString() / parse()	'14:30:00'
Instant	TEXT (ISO)	toString() / parse()	'2026-07-23T14:30:00Z'
BigDecimal	TEXT	toPlainString() / BigDecimal()	'125.50'
List<ItemTalaoDto>	TEXT (JSON)	kotlinx.serialization	[[...], {...}]

Decisao tecnica: Valores monetarios (iva, total, precoUnitario) sao guardados como TEXT, nao REAL. Isto evita perda de precisao de ponto flutuante do BigDecimal. A conversao faz-se com toPlainString() / BigDecimal().

9. Historico de migracoes (v1 a v5)

Versao	Nome	Descricao
1 -> 2	MIGRATION_1_2	Adiciona coluna textoOcr (TEXT NULL) a tabela talao
2 -> 3	MIGRATION_2_3	Adiciona 4 colunas a talao: nifCliente, serie, dataVencimento, metodoPagamento (todas TEXT NULL)
3 -> 4	MIGRATION_3_4	Cria tabela centro_custo (fref PK, nmfref, agnome)
4 -> 5	MIGRATION_4_5	Adiciona 4 colunas a talao: funcn, fref, nmfref, agnome (todas TEXT NOT NULL DEFAULT '')

SQL das migracoes

```
-- MIGRATION_1_2 (v1 -> v2) ALTER TABLE talao ADD COLUMN textoOcr TEXT -- MIGRATION_2_3 (v2 -> v3) ALTER
TABLE talao ADD COLUMN nifCliente TEXT ALTER TABLE talao ADD COLUMN serie TEXT ALTER TABLE talao ADD COLUMN
dataVencimento TEXT ALTER TABLE talao ADD COLUMN metodoPagamento TEXT -- MIGRATION_3_4 (v3 -> v4) CREATE
TABLE IF NOT EXISTS centro_custo ( fref TEXT NOT NULL PRIMARY KEY, nmfref TEXT NOT NULL, agnome TEXT NOT
NULL ) -- MIGRATION_4_5 (v4 -> v5) ALTER TABLE talao ADD COLUMN funcn TEXT NOT NULL DEFAULT '' ALTER TABLE
talao ADD COLUMN fref TEXT NOT NULL DEFAULT '' ALTER TABLE talao ADD COLUMN nmfref TEXT NOT NULL DEFAULT ''
ALTER TABLE talao ADD COLUMN agnome TEXT NOT NULL DEFAULT ''
```

IMPORTANTE: Todas as migracoes sao nao-destrutivas (apenas ADD COLUMN ou CREATE TABLE). Nenhuma migracao apaga dados existentes. A app executa as migracoes automaticamente ao abrir.

10. Fluxo de sincronizacao

Sincronizacao de centros de custo (FREF)

Passo	Descricao
1	App abre / utilizador toca 'Sincronizar'
2	HomeViewModel chama <code>frefRepository.sincronizar(serialDispositivo)</code>
3	FrefRepository cria ligacao JDBC via <code>RemoteDatabaseManager</code>
4	Executa SELECT na tabela <code>PHPRetailConcept.FREF</code> com filtro <code>ENCSERIE/SINCTAB</code>
5	Converte <code>ResultSet</code> para <code>List<CentroCustoEntity></code>
6	Executa <code>substituirTodos()</code> no Room (DELETE ALL + INSERT)
7	Devolve <code>ResultadoSync.Sucesso(total)</code> ou <code>ResultadoSync.Erro(mensagem)</code>
8	UI atualiza o chip de estado: LIGADO (verde) ou ERRO/OFFLINE (vermelho)

Estados de ligacao (EstadoBd)

Estado	Significado	UI
DESCONHECIDO	Ainda nao tentou sincronizar	Cinzento
A_SINCRONIZAR	Sincronizacao em curso	Amarelo (loading)
LIGADO	Ultima sincronizacao com sucesso	Verde
ERRO	Ultima sincronizacao falhou	Vermelho (OFFLINE)

Offline-first: Apos a primeira sincronizacao bem-sucedida, a app funciona completamente offline. Os centros de custo ficam em cache no Room. Os taloes sao guardados localmente e podem ser exportados via JSON/XML/CSV a qualquer momento.

11. Exportacao de dados

Cada talao guardado gera automaticamente 3 ficheiros de exportacao em `filesDir/receipts/`:

Formato	Ficheiro	Gerador	Conteudo
JSON	<code>talao_{id}.json</code>	<code>TalaoJsonDto (kotlinx.serialization)</code>	Todos os campos + itens
XML	<code>talao_{id}.xml</code>	<code>XmlExporter (manual, sem dep.)</code>	Todos os campos + itens
CSV	<code>talao_{id}.csv</code>	<code>CsvExporter (separador ';')</code>	1 linha por produto

CSV de captura (CapturaCsvExporter)

Alem do CSV por talao, a app gera um CSV de captura no momento da fotografia com metadata do dispositivo. Este CSV e enviado para o Google Drive junto com a imagem.

Coluna CSV	Descricao
SERIAL	Android ID do dispositivo
IDREG	ID de registo (AAAAMMDDHHMMSS)
TIPO	Tipo de captura (IMG)

GPS	Coordenadas GPS (lat,lon)
FUNCN	Numero do funcionario
FREF	Codigo do centro de custo
FUNCDESC	Nome do centro de custo (NMFREF)
ENCARREGADO	Abreviatura (AGNOME)
FUNOBS	Observacoes do funcionario
FORNECEDOR	Nome da empresa (do OCR/QR)
NIF_FORNECEDOR	NIF do fornecedor
NIF_CLIENTE	NIF do cliente
SERIE	Serie do documento
DATA	Data da fatura
DATA_VENCIMENTO	Data de vencimento
METODO_PAGAMENTO	Metodo de pagamento
QRCODE	Conteudo bruto do QR code AT
TIPODOC	Tipo de documento (FT, FS, FR...)
TOTAL	Valor total

12. Informacoes para a equipa de backend

Resumo do que a app espera do servidor

- **MariaDB acessivel na rede local da obra** - A app liga via JDBC na porta configurada (tipicamente 3306). Sem SSL atualmente.
- **Base de dados PHPRetailConcept** - Com tabela FREF acessivel pelo utilizador configurado.
- **Tabela FREF com colunas minimas:** FREF (PK), NMFREF, AGNOME, ENCSERIE, SINCTAB.
- **Utilizador com permissao SELECT** na tabela PHPRetailConcept.FREF.
- **Atribuicao de obras via ENCSERIE** - Inserir o Android ID do tablet (formato 'AID_XXXX') nos registos FREF que devem aparecer nesse tablet.

O que a app NAO faz (atualmente)

- Nao escreve dados no MariaDB (so le a tabela FREF).
- Nao sincroniza taloes para o servidor (ficam no tablet + Google Drive).
- Nao usa API REST (ligacao JDBC direta).
- Nao suporta SSL na ligacao JDBC (servidor precisa de configuracao).

Evolucoes futuras planeadas

- **API REST** - Substituir JDBC direto por endpoints REST (prioridade alta).
- **SSL no MariaDB** - Ativar SSL no servidor para encriptar o trafego.
- **Sincronizacao de taloes** - Enviar dados dos taloes para o servidor.
- **SQLCipher** - Encriptar a base de dados local no tablet.

Ficheiros de codigo relevantes

Ficheiro	Pacote	Funcao
ConfigManager.kt	core/config/	Le e descripta o XML de configuracao
AppConfig.kt	core/config/	Data class com credenciais descriptadas
RemoteDatabaseManager.kt	core/database/remote/	Cria ligacoes JDBC ao MariaDB
FrefRepository.kt	core/database/remote/	Sincroniza centros de custo (FREF)
AppDatabase.kt	core/database/	Definicao Room (entidades, DAOs, versao)
TalaoEntity.kt	core/database/entity/	Entidade Room da tabela talao
CentroCustoEntity.kt	core/database/entity/	Entidade Room da tabela centro_custo
TalaoDao.kt	core/database/dao/	Queries da tabela talao
CentroCustoDao.kt	core/database/dao/	Queries da tabela centro_custo
Migrations.kt	core/database/	Migracoes v1->v2, v2->v3, v3->v4, v4->v5
Converters.kt	core/database/converter/	TypeConverters (datas, BigDecimal, JSON)
DatabaseModule.kt	di/	Injecao Hilt do Room + migracoes
TalaoRepositoryImpl.kt	data/repository/	Guarda taloes + gera exportacoes

Contacto: Para questoes sobre a chave AES, formato do XML de configuracao, ou atribuicao de centros de custo a tablets, consultar a equipa de desenvolvimento mobile. A chave de encriptacao e a mesma que era utilizada no sistema Zebra anterior.